**Pokémon Team Builder — Take-Home Case Study**

Build an end-to-end Pokémon team-building application with a frontend UI, a backend API, tests, and storage. You may use any technologies, including AI.

---

**About Pokémon**

Pokémon is a franchise built around collectible creatures, each with a name, one or two elemental types (Fire, Water, Grass, Electric, etc. - 18 types), and base stats like HP and attack. Players build teams of up to 6 Pokémon and battle them against other teams.

Types have strengths and weaknesses against each other (e.g., Water beats Fire, Fire beats Grass). A well-built team usually has a mix of types, so it isn't easily countered by any single opposing type. A chart of type effectiveness can be found here: [https://pokemondb.net/type](https://pokemondb.net/type).

---

**PokéAPI Reference**

Base URL: `https://pokeapi.co/api/v2/` — full documentation at [https://pokeapi.co](https://pokeapi.co). Key endpoints:

- List all Pokémon: `https://pokeapi.co/api/v2/pokemon`
- Individual Pokémon detail: `https://pokeapi.co/api/v2/pokemon/{id}/`
  - Name: `name`
  - Types: `types[] → type → name`
  - Stats: `stats[]`
  - Moves: `moves[]`
  - Sprite: `sprites → front_default`

---

**Task 1: Team Builder**

1. Display all Pokémon in a responsive grid showing name, type(s), stats, and sprite image.
2. Allow users to build a team of up to 6 Pokémon. Order matters; users can add, remove, and reorder. Persist all team information so a user can access their teams across sessions.
3. Allow users to create, delete, and update multiple teams using the team builder UI.
4. Given a Pokémon team, create an optimal counter team of equal size. Open-ended. Show your creativity here!

---

**Task 2: Data Change Notifications**

PokéAPI data can change on rare occasions.

1. Create a backend job to scan for changes in any data used by the team builder.
2. If a user's Pokémon were involved in a change within the last week, display a frontend alert to that user describing what changed.
3. Prove that this feature works in a demo.

**Task 3: Technical Document**

1. Create a one-page README.md on key design decisions and assumptions, showing your technical acumen and communication skills.

---

**Deliverables**

- All code and readme in a shareable repository.
- Frontend hosted at a public URL.